

QUESTION 1

BritBooks has an orders table with one million rows.

A query searches for one customer but there is no index on customer_id .

Explain:

What the database may need to read

Why the query becomes slower as the table grows

One way to improve the query

Answer – Since the Index is not there on customer_id the query will read the full table row by row to filter the customer_id

As the table grows the number of rows and the data grows and to find customer_id it goes through all the table data which makes the query response slower

We can add Indexing on customer_id so that the query response will be faster as instead of reading the entire table row by row it just reads the customer_id column which makes the query faster

QUESTION 2

BritBooks frequently runs this query:

SQL

```
SELECT order_id, amount_gbp  
FROM orders  
WHERE customer_id = 101;
```

Answer the following:

Which column should be indexed?

How can the index improve the query?

What is one disadvantage of adding an index?

Answer – The customer_id column should be indexed

With Indexing the query look for only the customer_id column instead of doing full table scan making the query faster

Indexes use additional storage and when we need full table aggregations it will be a disadvantage

QUESTION 3

BritBooks wants to inspect how SQLite finds orders for one customer

REQUIRED TABLE

Column Data type

order_id INTEGER PRIMARY KEY

customer_id INTEGER NOT NULL

amount_gbp REAL NOT NULL

status TEXT NOT NULL

TASK

Create query_performance.db .

Create the orders table.

Insert 5,000 sample orders.

Use EXPLAIN QUERY PLAN before creating an index.

Create an index on customer_id .

Run EXPLAIN QUERY PLAN again.

Print both query plans.

Answer - Before Indexing the Query plan

```
1 CREATE TABLE orders(
2   order_id INT PRIMARY KEY,
3   customer_id INT NOT NULL,
4   amount_gbp REAL NOT NULL,
5   status TEXT NOT NULL
6 );
7 INSERT INTO orders(order_id,customer_id,amount_gbp,status) VALUES
8 (101,10,250,'Order completed'),
9 (102,20,550,'Order pending'),
10 (103,30,2850,'Order initiated'),
11 (104,40,2580,'Order completed'),
12 (105,50,2150,'Order completed');
13 EXPLAIN QUERY PLAN
14 SELECT * FROM orders WHERE customer_id=30;
```

id	parent	notused	detail
2	0	216	SCAN orders

After Indexing the Query plan

```
13 EXPLAIN QUERY PLAN
14 SELECT * FROM orders WHERE customer_id=30;
15
16 -- Indexing
17 CREATE INDEX idx_orders ON orders(customer_id);
18
19
20 --After indexing
21 EXPLAIN QUERY PLAN
22 SELECT * FROM orders WHERE customer_id=30;
```

id	parent	notused	detail
3	0	61	SEARCH orders USING INDEX idx_order...

QUESTION 4

BritBooks needs the total value of dispatched orders.

TASK

Create an SQLite script that compares these approaches:

Approach A

- Read every order.
- Filter Dispatched rows in Python.
- Calculate the total in Python.

Approach B

- Filter rows in SQL.
- Calculate the total using SUM .
- Return only the final value.

Print both totals and confirm that they match

```

1 CREATE TABLE orders(
2   order_id INT PRIMARY KEY,
3   customer_id INT NOT NULL,
4   amount_gbp REAL NOT NULL,
5   status TEXT NOT NULL
6 );
7 INSERT INTO orders(order_id,customer_id,amount_gbp,status) VALUES
8 (101,10,250,'Order completed'),
9 (102,20,550,'Dispatched'),
10 (103,30,2850,'Order initiated'),
11 (104,40,2580,'Dispatched'),
12 (105,50,2150,'Order completed');
13 SELECT SUM(amount_gbp) AS sql_total
14 FROM orders
15 WHERE status='Dispatched';

```

⌵ sql_total

3130

```

orders=[
    {"order_id":101,"customer_id":10,"amount_gbp":250,"status":"Order completed"},
    {"order_id":102,"customer_id":20,"amount_gbp":550,"status":"Dispatched"},
    {"order_id":103,"customer_id":30,"amount_gbp":2850,"status":"Order initiated"},
    {"order_id":104,"customer_id":40,"amount_gbp":2580,"status":"Dispatched"},
    {"order_id":105,"customer_id":50,"amount_gbp":2150,"status":"Order completed"}
]

```

```

def sum_of_orders():
    total=0
    for order in orders:
        if order["status"] == "Dispatched":
            total += order["amount_gbp"]
    print("python_total:",total)

```

sum_of_orders()

python total: 3130

Question 5 — MCQ: Full Table Scan M C Q

What happens during a full table scan?

- A. The database reads only one indexed row.
- B. The database checks every row in the table.
- C. The database deletes unused rows.
- D. The database compresses the table.

Answer – B

QUESTION 6

Which column is the best index candidate for this common query?

SQL

```
SELECT *  
FROM orders  
WHERE customer_id = 101;
```

- A. customer_id
- B. amount_gbp
- C. A column that is never queried
- D. Every column automatically

Answer - A

QUESTION 7

What is a common disadvantage of an index?

- A. It prevents all queries.
- B. It uses storage and adds write overhead.
- C. It removes primary keys.
- D. It converts rows into files.

Answer – B

QUESTION 8

Which query transfers the least unnecessary data?

- A. `SELECT * FROM orders;`
- B. `SELECT * FROM orders;` followed by Python filtering
- C. `SELECT order_id FROM orders WHERE status = 'Dispatched';`
- D. `SELECT * FROM customers, orders;`

Answer – C

QUESTION 9

An `order_status` column contains only Confirmed , Dispatched , and Cancelled . Why might an index on this column provide limited improvement for some queries?

- A. The column has very few distinct values.
- B. Text columns can never be indexed.
- C. Indexes work only on dates.
- D. Status values are stored in pounds.

Answer – A

QUESTION 10

BritBooks stores three years of order data.

Most reports filter by order date.

Explain:

- A suitable partition key

- A possible folder layout

- How partition pruning reduces the amount of data read

Answer – Suitable partition key is Order date

Folder layout will be like /orders/year=2026/month=01/

Partition pruning reduces the amount of data read by matching the folder instead of scanning every year and month.

Question 11

BritBooks has:

- 100 million orders per year

- Reports that normally filter by month

- Very few reports for a single hour

OPTION A

T E X T

/orders/year=2026/month=07/ OPTION

Option B

T E X T

/orders/year=2026/month=07/day=17/hour=10/

Answer - OPTION A

as Monthly filtering matches the normal query pattern. Hourly partitions would create many directories and could produce very small partitions

Question 12

A daily partition contains 10,000 files, each approximately 50 KB.

Explain:

Why opening many small files can be inefficient

What compaction means

When a compaction job could run

Answer -

Each file has opening and metadata overhead. Thousands of files can therefore take longer to list, open, and schedule than a smaller number of larger files.

Compaction combines many small files into fewer larger files.

BritBooks could run compaction after the daily load or during a quieter period.

Question 13

BritBooks receives three small order files for one day.

INPUT FILES

T E X T

orders_1.csv

orders_2.csv

orders_3.csv

Each file contains:

T E X T

order_id,customer_name,amount_gbp

TASK

Create a folder named small_order_files .

Create the three CSV files with one order in each.

Combine them into orders_2026-07-17.csv .

Write the header only once.

Print the number of combined rows.

Answer -

```
from pathlib import Path
import csv

input_folder = Path("small_order_files")
input_folder.mkdir(exist_ok=True)

files_and_rows = {
    "orders_1.csv": [1001, "Amelia Clarke", 34.99],
    "orders_2.csv": [1002, "Oliver Bennett", 18.50],
    "orders_3.csv": [1003, "Aisha Khan", 42.75]
}

header = ["order_id", "customer_name", "amount_gbp"]

# Write individual CSV files
for file_name, row in files_and_rows.items():
    with (input_folder / file_name).open(
        "w",
        newline="",
        encoding="utf-8"
    ) as file:
        writer = csv.writer(file)
        writer.writerow(header)
        writer.writerow(row)

# Combine all CSV files
output_file = Path("orders_2026-07-17.csv")
combined_rows = []

for input_file in sorted(input_folder.glob("orders_*.csv")):
```

```

for input_file in sorted(input_folder.glob("orders_*.csv")):
    with input_file.open(
        "r",
        newline="",
        encoding="utf-8"
    ) as file:
        reader = csv.DictReader(file)
        combined_rows.extend(reader)

# Write combined output file
with output_file.open(
    "w",
    newline="",
    encoding="utf-8"
) as file:
    writer = csv.DictWriter(file, fieldnames=header)
    writer.writeheader()
    writer.writerows(combined_rows)

print(f"Combined rows: {len(combined_rows)}")
print(f"Output file: {output_file}")

```

Combined rows: 3
 Output file: orders_2026-07-17.csv

QUESTION 14

Which is the best partition key when most queries filter orders by month?

- A. Customer name
- B. Order month
- C. Product description
- D. Order amount rounded to the nearest pound

Answer - B

QUESTION 15

What does partition pruning do?

- A. Deletes old partitions automatically
- B. Reads only partitions that match the filter
- C. Creates an index on every column
- D. Converts CSV into JSON

Answer – B

QUESTION 16

What is a possible result of partitioning data too finely?

- A. Too many small folders and files
- B. All rows become duplicates
- C. SQL stops supporting filters
- D. Compression becomes impossible

Answer – A

QUESTION 17

Why are thousands of tiny files often inefficient?

- A. They cannot contain numbers.
- B. Each file adds metadata and opening overhead.
- C. They always delete their schemas.
- D. Small files cannot be stored in folders.

Answer - B

QUESTION 18

What is compaction?

- A. Splitting one file into thousands of files
- B. Combining small files into fewer larger files

- C. Removing all partition columns
- D. Replacing SQL with Python

Answer – B

QUESTION 19

Complete the table using short phrases.

Format	Text or binary?	Row or column oriented?	Common use CSV
CSV	TEXT	ROW	Simple exchange and manual inspection
Parquet	BINARY	Columnar	Data lake and analytics workloads
Avro	BINARY	ROW	Data exchange and streaming evnts
ORC	BINARY	Columnar	Hive and Hadoop analytics

QUESTION 20

BritBooks has an analytics file with 80 columns.

A report needs only:

order_date
product_id
amount_gbp

Explain why a columnar format may read less data than CSV.

Answer -

CSV stores complete rows, so a reader normally processes data from every column in each row. A columnar format stores values from the same column together. An analytics engine can therefore read only the three required columns.

QUESTION 21

Choose the most suitable option and give one reason.

Requirement	Choose from
A small file that a non-technical user can open easily	CSV / Parquet
Large analytical data where queries select a few columns	CSV / Parquet
Streaming events with an explicit schema	Avro / CSV
Large Hive analytics tables	ORC / CSV

Answer -

Requirement	Choice	Reason
Easy manual opening of CSV files	CSV	It is plain text, human-readable, and widely supported across tools.
Large analytical datasets	Parquet	It is columnar, highly compressed, and optimized for analytical queries.
Streaming events with schema	Avro	It stores structured records with a defined schema, ideal for streaming pipelines.
Hive analytics tables	ORC	It is a columnar format designed for large analytical workloads and integrates well with Hive.

QUESTION 22

BritBooks sends order events between systems

Field	Type	Description / Notes
order_id	string	Unique order identifier
customer_id	string	Unique customer identifier
amount_gbp	double	Order amount in GBP
order_status	string	Status of the order
coupon_code	string or null	Optional; defaults to null if not present

TASK Create the schema as a Python dictionary. Write it to order_event_schema.json . Use indentation to keep the JSON readable. Read the file again. Print the field names.

Answer -

```
import json
schema = {
    "type": "record",
    "name": "OrderEvent",
    "fields": [
        {"name": "order_id", "type": "string"},
        {"name": "customer_id", "type": "string"},
        {"name": "amount_gbp", "type": "double"},
        {"name": "order_status", "type": "string"},
        {
            "name": "coupon_code",
            "type": ["null", "string"],
            "default": None
        }
    ]
}
with open(
    "order_event_schema.json",
    "w",
    encoding="utf-8"
) as file:
    json.dump(schema, file, indent=2)
with open(
    "order_event_schema.json",
    "r",
    encoding="utf-8"
) as file:
    loaded_schema = json.load(file)
field_names = [
    field["name"]
```

```
with open(
    "order_event_schema.json",
    "r",
    encoding="utf-8"
) as file:
    loaded_schema = json.load(file)
    field_names = [
        field["name"]
        for field in loaded_schema["fields"]
    ]
    print(field_names)

['order_id', 'customer_id', 'amount_gbp', 'order_status', 'coupon_code']
```

QUESTION 23

Which statement about CSV is correct?

- A. It always stores a strict schema.
- B. It is a human-readable text format.
- C. It is a columnar binary format.
- D. It automatically supports nested objects.

Answer - B

QUESTION 24

Why is Parquet commonly used for analytics?

- A. It stores every value as plain text.
- B. It is columnar and can read selected columns.
- C. It supports only one data type.
- D. It cannot be compressed.

Answer – B

QUESTION 25

Which use case best matches Avro?

- A. A simple spreadsheet opened manually
- B. Structured event exchange with schema evolution
- C. Image storage
- D. Graph traversal

Answer – B

QUESTION 26

ORC is mainly associated with:

- A. Columnar analytics in the Hadoop and Hive ecosystem
- B. Storing passwords
- C. Editing source code
- D. Relational transactions

Answer – A

QUESTION 27

What is the main benefit of compression?

- A. It always removes CPU usage.
- B. It reduces storage size and data read from disk.
- C. It removes the need for a schema.
- D. It converts columns into rows.

Answer – B

QUESTION 28

BritBooks needs a daily sales report.

The report:

- Filters by order month

- Groups by product

- Reads only order date, product ID, quantity, and amount

- Uses data stored for five years

Identify:

- The best partition field

- A suitable analytical file format

- The columns the query should select

- One reason to avoid `SELECT *`

Answer -

- Partition using order date, such as year and month.

- Use Parquet or ORC for analytical storage.

- Select only order date, product ID, quantity, and amount.

- `SELECT *` reads unnecessary columns and increases input/output.

QUESTION 29

BritBooks has these datasets:

- Raw order exports

- Cleaned analytical orders

- Streaming click events

- Monthly sales summaries

TASK

Create a table with:

Dataset	File format	Partition key	Reason
---------	-------------	---------------	--------

Use CSV, Parquet, Avro, or ORC where appropriate.

Dataset	File Format	Partition Key	Reason
Raw order exports	CSV	Ingestion date	Keeps original portable source files; easy to inspect manually.
Cleaned analytical orders	Parquet	Order year and month	Columnar format enables fast analytics and partition pruning.
Streaming click events	Avro	Event date and hour	Ideal for structured event records and supports schema evolution.
Monthly sales summaries	Parquet or ORC	Summary year and month	Optimized for fast analytical reads in BI and data warehouse workloads.

QUESTION 30

BritBooks wants to organise orders by year and month.

DATA

TEXT

1001, 2026-06-28, £34.99

1002, 2026-07-02, £18.50

1003, 2026-07-17, £42.75

TASK

Create an orders_lake folder.

Create year=YYYY/month=MM folders from each date.

Write each order to an orders.csv file in the correct partition.

Include the CSV header.

Print the created file paths.

ANSWER -

```

from pathlib import Path
from datetime import datetime
import csv

orders = [
    {
        "order_id": 1001,
        "order_date": "2026-06-28",
        "amount_gbp": 34.99
    },
    {
        "order_id": 1002,
        "order_date": "2026-07-02",
        "amount_gbp": 18.50
    },
    {
        "order_id": 1003,
        "order_date": "2026-07-17",
        "amount_gbp": 42.75
    }
]

base_folder = Path("orders_lake")
header = ["order_id", "order_date", "amount_gbp"]

# Write partitioned files
for order in orders:
    order_date = datetime.strptime(order["order_date"], "%Y-%m-%d")

```

```

    partition_folder = (
        base_folder
        / f"year={order_date.year}"
        / f"month={order_date.month:02d}"
    )

    partition_folder.mkdir(parents=True, exist_ok=True)

    output_file = partition_folder / "orders.csv"
    file_exists = output_file.exists()

    with output_file.open("a", newline="", encoding="utf-8") as file:
        writer = csv.DictWriter(file, fieldnames=header)
        if not file_exists:
            writer.writeheader()
        writer.writerow(order)

# Print all generated CSV paths
for file_path in sorted(base_folder.rglob("orders.csv")):
    print(file_path)

```

```

orders_lake\year=2026\month=06\orders.csv
orders_lake\year=2026\month=07\orders.csv

```

QUESTION 31

BritBooks stores:

- Five years of orders in one uncompressed CSV file
- 100 columns in every row
- Reports that normally use four columns
- No date partitions
- One daily process that appends thousands of tiny files

QUESTION

Recommend four improvements.

Answer -

Convert cleaned analytics data to Parquet or ORC.

Partition data by a commonly filtered date field.

Select only required columns in queries.

Compact the daily small files into fewer larger files.

Use compression appropriate for the chosen analytical format.

QUESTION 32

Which format is the strongest default choice for large analytical order data?

- A. Parquet
- B. Plain text without separators
- C. Image files
- D. HTML

Answer – A

QUESTION 33

Which filter is most likely to support pruning for this layout?

TEXT

/orders/year=2026/month=07/

- A. WHERE customer_name = 'Amelia Clarke'
- B. WHERE year = 2026 AND month = 7
- C. WHERE amount_gbp > 20
- D. WHERE status = 'Confirmed'

Answer – B

QUESTION 34

What does projection mean in query optimisation?

- A. Selecting only required columns
- B. Deleting old rows
- C. Creating additional small files
- D. Sorting every table

Answer – A

QUESTION 35

Which format is a suitable choice for structured streaming event records?

- A. Avro
- B. Bitmap image
- C. PDF
- D. Plain HTML

Answer – A

QUESTION 36

Which combination is most suitable for monthly sales analytics?

- A. One large CSV, SELECT * , and no date filter
- B. Parquet, date partitions, early filtering, and required columns only
- C. Thousands of tiny text files and no compression
- D. Graph storage for every sales total

Answer - B